

Informe del Proyecto: Simulador de Sistemas de Archivos (FAT y Ext2)

Bridigo Noguera
Angel Besteiro

October 9, 2025

Contents

1	Introducción	2
2	Definición de Sistemas de Archivos Simulados	2
2.1	File Allocation Table (FAT)	2
2.2	Second Extended Filesystem (Ext2)	2
3	Análisis del Código por Archivo	3
3.1	Archivo <code>proyecto.h</code> (Definiciones)	3
3.2	Archivo <code>proyecto.cpp</code> (Implementación)	3
3.2.1	Lógica de FAT (<code>FATFileSystem</code>)	3
3.2.2	Lógica de Ext2 (<code>Ext2FileSystem</code>)	4
4	Conclusión	5

1 Introducción

Este proyecto consiste en un simulador básico de sistemas de archivos implementado en C++. Permite al usuario interactuar con un disco virtual y probar las operaciones fundamentales (crear, leer, eliminar) en dos arquitecturas de gestión de archivos distintas: **File Allocation Table (FAT)** y **Second Extended Filesystem (Ext2)**.

2 Definición de Sistemas de Archivos Simulados

2.1 File Allocation Table (FAT)

FAT es uno de los sistemas de archivos más antiguos y sencillos, desarrollado por Microsoft.

- **Lógica Clave:** Utiliza una tabla maestra (**FAT Table**) almacenada al inicio del disco para gestionar el espacio.
- **Asignación de Archivos:** Un archivo se representa como una **lista enlazada de bloques**. El directorio solo guarda el índice del **primer bloque**. La tabla FAT guarda el índice del **siguiente bloque** en la cadena, hasta llegar a un marcador de Fin de Archivo (EOF).
- **Simulación en Código:** La tabla `fat_table` es un vector de enteros, donde el índice del vector es el bloque y el valor es el siguiente bloque.

2.2 Second Extended Filesystem (Ext2)

Ext2 fue el sistema de archivos estándar de Linux por muchos años. Su diseño se enfoca en la robustez y la eficiencia en la gestión de archivos grandes mediante el uso de una estructura de metadatos más compleja.

- **Lógica Clave:** Utiliza **Inodos** (Index Nodes) para almacenar los metadatos de un archivo (tamaño, permisos, fechas) y, crucialmente, los **punteros** a sus bloques de datos.
- **Asignación de Archivos:** El inodo utiliza un esquema de punteros jerárquico:
 1. **Punteros Directos:** Apuntan directamente a los primeros bloques de datos.
 2. **Puntero Indirecto Simple:** Apunta a un **bloque intermedio** que, a su vez, contiene una lista de punteros a bloques de datos.
 3. Punteros Doble y Triple Indirecto (Omitidos en esta simulación).
- **Simulación en Código:** Se utiliza un vector de estructuras `Inode` (`inode_table`) y los archivos acceden a sus datos a través de los campos `direct_pointers` y `single_indirect`.

3 Análisis del Código por Archivo

3.1 Archivo proyecto.h (Definiciones)

Este archivo establece la arquitectura del simulador.

Listing 1: Extracto de constantes y estructuras en proyecto.h

```
1  const int DISK_BLOCKS = 256; // Nmero total de bloques en nuestro disco virtual.
2  const int BLOCK_SIZE = 64; // Tamao de cada bloque en bytes.
3  const int FAT_EOF = -1; // Marcador de Fin de Archivo para la tabla FAT.
4  const int FAT_FREE = 0; // Marcador de bloque libre para la tabla FAT.
5  const int NUM_DIRECT_POINTERS = 4; // Nmero de punteros directos en un inodo
   ext2.
6  // ...
7  struct Inode {
8      bool is_used = false;
9      int size = 0;
10     std::vector<int> direct_pointers; // Punteros directos a bloques de datos.
11     int single_indirect = -1; // Puntero a un bloque que contiene ms punteros.
12     // ...
13 };
14 // ...
15 class FileSystem {
16 public:
17     std::shared_ptr<Disk> disk;
18     // ... Mtodos virtuales puros ...
19     virtual bool create(const std::string& filename, const std::string& content)
20         = 0;
21     // ...
22 };
```

- **Líneas 6-10:** Definen las **constantes fundamentales** del disco virtual (256 bloques de 64 bytes) y los marcadores lógicos de FAT (FAT_EOF, FAT_FREE).
- **Línea 33:** La estructura `Inode` es clave para Ext2, conteniendo metadatos y punteros.
- **Línea 49:** Se define la **clase base abstracta** `FileSystem`, que utiliza **polimorfismo** para obligar a FAT y Ext2 a implementar las operaciones básicas.

3.2 Archivo proyecto.cpp (Implementación)

3.2.1 Lógica de FAT (FATFileSystem)

La gestión de archivos se centra en la `fat_table` para enlazar los bloques.

Listing 2: `FATFileSystem::create` - Enlace de bloques

```
22     // Enlaza los bloques en la tabla FAT.
23     for (size_t i = 0; i < free_blocks.size() - 1; ++i) {
```

```

24     fat_table[free_blocks[i]] = free_blocks[i + 1];
25 }
26 fat_table[free_blocks.back()] = FAT_EOF; // El ltimo bloque apunta a Fin de
    Archivo.
27 // ...
28 // Registra el archivo en el directorio.
29 directory[filename] = free_blocks[0];

```

- **Líneas 22-25:** Esta es la **lógica central de FAT**. Los bloques libres se encadenan secuencialmente. El índice i de la `fat_table` (que es un bloque de datos) se le asigna como valor el índice del siguiente bloque ($i + 1$), formando una **lista enlazada**. El último bloque se marca con `FAT_EOF`.
- **Línea 30:** El `directory` (directorio) solo necesita almacenar el índice del **primer bloque** de la cadena para poder acceder a todo el archivo.

Listing 3: FATFileSystem::read - Recorrido de la cadena

```

36 // Sigue la cadena en la tabla FAT hasta llegar al final.
37 while (current_block != FAT_EOF) {
38     content += disk->readBlock(current_block);
39     current_block = fat_table[current_block];
40 }

```

- **Líneas 37-40:** La función de lectura (y la de eliminación) recorre la `fat_table` de bloque en bloque, leyendo el contenido y actualizando `current_block` con el valor encontrado en la tabla. El bucle termina al encontrar `FAT_EOF`.

3.2.2 Lógica de Ext2 (Ext2FileSystem)

La gestión de archivos se centra en la estructura `Inode` para la asignación de punteros.

Listing 4: Ext2FileSystem::create - Asignación de punteros

```

108 // 1. Llenar punteros directos.
109 for (int i = 0; i < NUM_DIRECT_POINTERS && blocks_assigned < blocks_needed;
    ++i) {
110     inode.direct_pointers[i] = free_blocks[blocks_assigned];
111     blocks_assigned++;
112 }
113
114 // 2. Llenar puntero indirecto simple (si es necesario).
115 if (blocks_assigned < blocks_needed) {
116     // ... Lgica para reservar el bloque indirecto ...
117     inode.single_indirect = indirect_block_idx;
118
119     std::string pointer_data;
120     for (int i = blocks_assigned; i < blocks_needed; ++i) {
121         int block_ptr = free_blocks[i];
122         pointer_data.append(reinterpret_cast<const char*>(&block_ptr),
            sizeof(int));
    }
}

```

```

123     }
124     disk->writeBlock(indirect_block_idx, pointer_data);
125 }

```

- **Líneas 109-113:** Se llenan los **punteros directos** (`NUM_DIRECT_POINTERS = 4`). Esto permite el acceso rápido a los bloques de archivos pequeños.
- **Líneas 116-126:** Si se necesitan más bloques, se utiliza el **puntero indirecto simple**. Se reserva un bloque (`indirect_block_idx`) cuyo contenido no son datos de archivo, sino una secuencia de **punteros** (enteros) a los bloques de datos restantes.

Listing 5: Ext2FileSystem::read - Desreferenciación indirecta

```

140 // 2. Leer desde puntero indirecto simple.
141 if (inode.single_indirect != -1) {
142     std::string pointer_block_content =
143         disk->readBlock(inode.single_indirect);
144     for(size_t i = 0; i < pointer_block_content.length(); i += sizeof(int)) {
145         int block_ptr;
146         std::memcpy(&block_ptr, pointer_block_content.c_str() + i,
147             sizeof(int));
148         content += disk->readBlock(block_ptr);
149     }
150 }

```

- **Líneas 141-147:** Esta es la **lógica multinivel** de Ext2. Primero se lee el bloque de punteros (`inode.single_indirect`). Luego, su contenido se itera, extrayendo cada 4 bytes (`sizeof(int)`) para obtener el índice de un bloque de datos real, el cual se lee a continuación.

4 Conclusión

El proyecto es un simulador pedagógico exitoso que ilustra las diferencias fundamentales entre la gestión de archivos basada en tablas (FAT) y la gestión basada en inodos (Ext2).

- **FAT:** Fácil de implementar, ideal para archivos pequeños, pero con fragmentación potencial y limitaciones de tamaño. La lógica es una simple **cadena de bloques**.
- **Ext2:** Más complejo, pero eficiente para archivos grandes gracias al esquema de **punteros directos e indirectos**, que reduce la longitud de la cadena de búsqueda.